

ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ №1

Варіант №1 — «Система обліку студентів»

Виконав: Левандівський Назар-Генрі

Група: Фел-31

ЛАБОРАТОРНА РОБОТА №1

Організація професійного Python-проєкту та Git

1. Мета роботи

Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл `pyproject.toml`, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

У результаті виконання лабораторної роботи студент повинен уміти:

створювати та активувати `virtual environment`;

створювати структурований Python-проєкт;

використовувати `src-layout`;

створювати Python package;

розділяти програму на модулі;

використовувати конструкцію `if __name__ == "__main__":`;

створювати файл `pyproject.toml`;

створювати `.gitignore`;

оформлювати `README.md`;

ініціалізувати Git repository;

виконувати `git add`, `git commit`, `git status`, `git log`;

створювати окремі commits для логічних етапів розробки;

запускати програму як Python module;

дотримуватися базових правил PEP 8.

2. Теоретичні відомості

2.1. Від скрипта до Python-проєкту

Невелика навчальна програма часто складається з одного файла:

program.py

Для професійної розробки така організація швидко стає незручною. Зі збільшенням кількості функцій, класів, тестів і зовнішніх залежностей програму необхідно поділяти на логічні компоненти.

Професійний Python-проєкт може мати таку структуру:

```
student_project/  
|  
├── pyproject.toml  
├── README.md  
├── .gitignore  
|  
├── src/  
|   ├── student_project/  
|   |   ├── __init__.py  
|   |   ├── main.py  
|   |   ├── models.py  
|   |   └── services.py  
|  
└── tests/  
    └── test_services.py
```

Каталог src містить програмний код.

Каталог tests призначений для автоматизованих тестів.

Файл README.md описує проєкт.

Файл `pyproject.toml` містить метадані та конфігурацію Python-проєкту.

Файл `.gitignore` визначає файли, які не потрібно додавати до Git repository.

3. Virtual environment

Virtual environment дозволяє ізолювати бібліотеки одного Python-проєкту від іншого.

Створення середовища:

```
python -m venv .venv
```

Активація у Windows:

```
.venv\Scripts\activate
```

Linux/macOS:

```
source .venv/bin/activate
```

Перевірити Python:

```
python --version
```

Перевірити встановлені packages:

```
python -m pip list
```

Virtual environment `.venv` не потрібно додавати до Git repository.

4. Python modules та packages

Python module - це звичайний `.py` файл.

Наприклад:

services.py

Python package - каталог із Python modules.

Наприклад:

student_project/

__init__.py

main.py

services.py

Модулі можна імпортувати:

```
from student_project.services import calculate_average
```

Такий поділ дозволяє відокремити:

введення/виведення;

обчислення;

data models;

business logic;

configuration;

tests.

5. Точка входу програми

Рекомендовано відокремлювати визначення функцій від запуску application.

```
def main() -> None:
```

```
    print("Application started")
```

```
if __name__ == "__main__":
```

```
    main()
```

Конструкція:

```
if __name__ == "__main__":
```

означає, що `main()` буде виконано тільки тоді, коли файл запускається безпосередньо.

6. Файл `pyproject.toml`

Приклад:

```
[build-system]
```

```
requires = ["setuptools>=70"]
```

```
build-backend = "setuptools.build_meta"
```

```
[project]
```

```
name = "student-project"
```

```
version = "0.1.0"
```

```
description = "Laboratory work for Professional Python"
```

```
requires-python = ">=3.11"
```

Для встановлення локального package можна використати:

```
python -m pip install -e .
```

Параметр `-e` означає `editable installation`.

7. Git

Git - distributed version control system, яка дозволяє:

зберігати історію змін;

повертатися до попередніх версій;

працювати в команді;

створювати branches;

відстежувати авторство змін;

зберігати проєкт у GitHub, GitLab або іншому repository hosting service.

Ініціалізація repository:

```
git init
```

Перегляд стану:

```
git status
```

Додавання файлів:

```
git add .
```

Створення commit:

```
git commit -m "Initial project structure"
```

Перегляд історії:

```
git log --oneline
```

Для лабораторної роботи рекомендується зробити щонайменше 3 логічні commits.

Наприклад:

```
Initial project structure
```

```
Add application logic
```

Add README and project configuration

8. Файл .gitignore

Приклад:

.venv/

__pycache__/

*.pyc

.pytest_cache/

.idea/

.vscode/

dist/

build/

*.egg-info/

.env

.gitignore допомагає не додавати до repository:

virtual environment;

temporary files;

caches;

IDE configuration;

build artifacts;

secrets.

9. README.md

Приклад:

Student Project

Навчальний проєкт з дисципліни «Професійний Python».

Requirements

Python 3.11+

Installation

python -m venv .venv

python -m pip install -e .

Run

python -m student_project.main

Author

Прізвище Ім'я

Група

10. Постановка лабораторної роботи

Студент повинен створити невеликий консольний Python-застосунок відповідно до свого індивідуального варіанта.

Програма повинна:

мати структуру src-layout;

містити щонайменше три Python modules;

містити щонайменше одну data model;

містити окремий module з business logic;

містити main.py;

використовувати type hints;

містити щонайменше три функції;

приймати декілька вхідних значень;

виконувати обробку даних;

формувати зрозумілий результат;

використовувати конструкцію if __name__ == "__main__":;

мати README.md;

мати .gitignore;

мати pyproject.toml;

бути доданою до Git repository.

11. Індивідуальні варіанти

Варіант 1. Система обліку студентів

Створити application для роботи зі студентами.

Для кожного студента зберігати:

ім'я;

прізвище;

групу;

середній бал.

Реалізувати:

додавання студента;

виведення студентів;

пошук за групою;

визначення студента з найбільшим середнім балом;

обчислення середнього бала групи.

Варіант 2. Система обліку книг

Для книги зберігати:

назву;

автора;

рік видання;

кількість сторінок.

Реалізувати:

додавання книги;

пошук за автором;

фільтрацію за роком;

визначення найбільшої книги;

обчислення середньої кількості сторінок.

Варіант 3. Інтернет-магазин

Для товару зберігати:

назву;

категорію;

ціну;

кількість.

Реалізувати:

додавання товару;

обчислення вартості залишків;

пошук товару;

фільтрацію за категорією;

визначення найдорожчого товару.

Варіант 4. Система банківських рахунків

Для рахунку зберігати:

ім'я клієнта;

номер рахунку;

баланс.

Реалізувати:

поповнення;

списання;

перевірку балансу;

пошук рахунку;

обчислення загальної суми коштів.

Не дозволяти отримувати від'ємний баланс.

Варіант 5. Каталог автомобілів

Для автомобіля:

марка;

модель;

рік;

ціна;

пробіг.

Реалізувати:

пошук за маркою;

фільтрацію за роком;

визначення найдорожчого автомобіля;

середню ціну;

автомобіль із найменшим пробігом.

Варіант 6. Облік працівників

Для працівника:

ім'я;

посада;

відділ;

зарплата.

Реалізувати:

список працівників;

пошук за відділом;

середню зарплату;

працівника з максимальною зарплатою;

сумарний фонд оплати праці.

Варіант 7. Система оцінювання студентів

Зберігати:

ПБ;

дисципліну;

оцінки.

Реалізувати:

середній бал;

мінімальну та максимальну оцінки;

визначення успішності;

сортування студентів;

пошук студентів із середнім балом вище заданого.

Варіант 8. Система бронювання готелю

Для номера:

номер;

тип;

ціна;

статус бронювання.

Реалізувати:

список вільних номерів;

бронювання;

скасування бронювання;

пошук найдешевшого номера;

обчислення можливої виручки.

Варіант 9. Облік спортивних результатів

Для спортсмена:

ім'я;

вид спорту;

результат;

категорія.

Реалізувати:

додавання результату;

пошук за видом спорту;

визначення найкращого результату;

середній результат;

формування рейтингу.

Варіант 10. Система обліку складських запасів

Для позиції:

код;

назва;

кількість;

ціна.

Реалізувати:

надходження;

списання;

пошук;

загальну вартість складу;

визначення товарів із низьким запасом.

Варіант 11. Облік замовлень ресторану

Для страви:

назва;

категорія;

ціна.

Для замовлення:

номер;

список страв.

Реалізувати:

створення замовлення;

додавання страв;

підрахунок суми;

найдорожчу позицію;

середню вартість замовлення.

Варіант 12. Фінансовий трекер

Для операції:

дата;

категорія;

сума;

тип income/expense.

Реалізувати:

додавання операції;

обчислення доходів;

обчислення витрат;

баланс;

витрати за категоріями.

Варіант 13. Система обліку комп'ютерної техніки

Для пристрою:

інвентарний номер;

тип;

виробник;

модель;

вартість.

Реалізувати:

пошук;

фільтрацію за виробником;

середню вартість;

найдорожчий пристрій;

загальну вартість обладнання.

Варіант 14. Система керування завданнями

Для task:

назва;

опис;

priority;

status.

Реалізувати:

створення task;

зміну status;

фільтрацію;

пошук;

підрахунок виконаних і невиконаних завдань.

Варіант 15. Облік результатів наукових експериментів

Для experiment:

назва;

researcher;

параметр;

результат;

дата.

Реалізувати:

додавання experiment;

пошук за researcher;

середнє значення результатів;

minimum/maximum;

фільтрацію результатів за заданим діапазоном.

12. Приклад виконання лабораторної роботи

Розглянемо варіант 1 — «Система обліку студентів».

Структура:

student_manager/

|

├─ pyproject.toml

├─ README.md

├─ .gitignore

|

├─ src/

| └─ student_manager/

| ├─ __init__.py

| ├─ main.py

| ├─ models.py

| └─ services.py

```
|  
└─ tests/
```

13. Файл models.py

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Student:
```

```
    first_name: str
```

```
    last_name: str
```

```
    group: str
```

```
    average_grade: float
```

```
@property
```

```
def full_name(self) -> str:
```

```
    return f"{self.first_name} {self.last_name}"
```

У класі Student використано dataclass, яка автоматично створює constructor та інші стандартні методи.

14. Файл services.py

```
from student_manager.models import Student
```

```
def calculate_group_average(  
    students: list[Student],
```

```
) -> float:
```

```
    if not students:
```

```
        return 0.0
```

```
    total = sum(
```

```
        student.average_grade
```

```
        for student in students
```

```
    )
```

```
    return total / len(students)
```

```
def find_best_student(
```

```
    students: list[Student],
```

```
) -> Student | None:
```

```
    if not students:
```

```
        return None
```

```
    return max(
```

```
        students,
```

```
        key=lambda student: student.average_grade,
```

```
    )
```

```
def filter_by_group(
```

```
    students: list[Student],
```

```
        group: str,  
    ) -> list[Student]:  
  
        return [  
            student  
            for student in students  
            if student.group == group  
        ]
```

```
def find_student(  
    students: list[Student],  
    last_name: str,  
) -> Student | None:
```

```
    for student in students:  
        if student.last_name.lower() == last_name.lower():  
            return student  
  
    return None
```

15. Файл main.py

```
from student_manager.models import Student  
from student_manager.services import (  
    calculate_group_average,  
    filter_by_group,
```

```
    find_best_student,  
    find_student,  
)
```

```
def create_demo_students() -> list[Student]:
```

```
    return [  
        Student(  
            first_name="Anna",  
            last_name="Koval",  
            group="PI-21",  
            average_grade=91.5,  
        ),  
        Student(  
            first_name="Oleh",  
            last_name="Melnyk",  
            group="PI-21",  
            average_grade=84.0,  
        ),  
        Student(  
            first_name="Iryna",  
            last_name="Bondar",  
            group="PI-22",  
            average_grade=95.2,  
        ),
```

```
Student(  
    first_name="Andrii",  
    last_name="Shevchenko",  
    group="PI-21",  
    average_grade=88.7,  
),  
]
```

```
def print_students(  
    students: list[Student],  
) -> None:
```

```
    print("\nStudents:")
```

```
    for student in students:  
        print(  
            f"{student.full_name:25}"  
            f"{student.group:10}"  
            f"{student.average_grade:6.2f}"  
        )
```

```
def main() -> None:
```

```
    students = create_demo_students()
```

```
print_students(students)
```

```
group = "PI-21"
```

```
group_students = filter_by_group(  
    students,  
    group,  
)
```

```
print(  
    f"\nStudents of group {group}:"  
)
```

```
print_students(group_students)
```

```
average = calculate_group_average(  
    group_students,  
)
```

```
print(  
    f"\nGroup average: {average:.2f}"  
)
```

```
best = find_best_student(students)
```

```
if best is not None:

    print(

        "\nBest student:",

        best.full_name,

        f"({best.average_grade:.2f})",

    )
```

```
student = find_student(

    students,

    "Melnik",

)
```

```
if student is not None:

    print(

        "\nStudent found:",

        student.full_name,

        student.group,

    )
```

```
if __name__ == "__main__":

    main()
```

16. Приклад роботи програми

Students:

Anna Koval	PI-21	91.50
------------	-------	-------

Oleh Melnyk	PI-21	84.00
Iryna Bondar	PI-22	95.20
Andrii Shevchenko	PI-21	88.70

Students of group PI-21:

Anna Koval	PI-21	91.50
Oleh Melnyk	PI-21	84.00
Andrii Shevchenko	PI-21	88.70

Group average: 88.07

Best student: Iryna Bondar (95.20)

Student found: Oleh Melnyk PI-21

17. Файл init.py

```
"""Student manager package."""
```

```
__version__ = "0.1.0"
```

18. pyproject.toml

```
[build-system]
```

```
requires = ["setuptools>=70"]
```

```
build-backend = "setuptools.build_meta"
```

```
[project]
name = "student-manager"
version = "0.1.0"
description = "Student management laboratory project"
requires-python = ">=3.11"
```

```
[project.scripts]
student-manager = "student_manager.main:main"
```

19. Встановлення проєкту

У кореневому каталозі:

```
python -m venv .venv
```

Активувати environment.

Після цього:

```
python -m pip install -e .
```

Запуск як module:

```
python -m student_manager.main
```

або через console command:

```
student-manager
```

20. .gitignore

```
.venv/
```

```
__pycache__/
```

```
*.pyc
```

```
.pytest_cache/
```

```
.idea/  
.vscode/  
dist/  
build/  
*.egg-info/  
.env
```

21. README.md

Student Manager

Laboratory project for the course

"Professional Python".

Description

Console application for managing students
and calculating academic statistics.

Requirements

Python 3.11+

Installation

```
python -m venv .venv
```

Activate the virtual environment.

```
python -m pip install -e .
```

Run

```
python -m student_manager.main
```

or

```
student-manager
```

Project structure

```
src/student_manager/models.py
```

Data models.

```
src/student_manager/services.py
```

Business logic.

```
src/student_manager/main.py
```

Application entry point.

Author

Student: Name Surname

Group: XX-00

22. Робота з Git

Перейти до каталогу проєкту:

```
git init
```

Перевірити стан:

```
git status
```

Перший commit:

```
git add .gitignore pyproject.toml
```

```
git add src/student_manager/__init__.py
```

```
git commit -m "Create project structure"
```

Другий commit:

```
git add src/student_manager/models.py
```

```
git add src/student_manager/services.py
```

```
git commit -m "Add student model and business logic"
```

Третій commit:

```
git add src/student_manager/main.py
```

```
git add README.md
```

```
git commit -m "Add application entry point and documentation"
```

Переглянути історію:

```
git log --oneline
```

Приклад:

```
8e5d011 Add application entry point and documentation
```

5a849a2 Add student model and business logic

12f11ac Create project structure

23. Завдання підвищеної складності

Для отримання максимальної оцінки студент повинен додатково реалізувати щонайменше дві можливості:

введення даних із клавіатури;

menu-driven interface;

validation input;

сортування;

пошук за декількома критеріями;

статистичні показники;

окрему configuration constant;

власний exception;

використання dataclass;

README з прикладом запуску програми.

24. Приклад консольного меню

```
def print_menu() -> None:
```

```
    print()
```

```
    print("1. Show all students")
```

```
    print("2. Find student")
```

```
    print("3. Show group")
```

```
    print("4. Show best student")
```

```
    print("5. Exit")
```

```
def run_menu(  
    students: list[Student],  
    ) -> None:  
  
    while True:  
  
        print_menu()  
  
        command = input(  
            "Select command: "  
        ).strip()  
  
        if command == "1":  
            print_students(students)  
  
        elif command == "2":  
  
            last_name = input(  
                "Last name: "  
            )  
  
            student = find_student(  
                students,  
                last_name,  
            )
```

```
if student:
    print(
        student.full_name,
        student.group,
        student.average_grade,
    )
```

```
else:
    print(
        "Student not found."
    )
```

```
elif command == "3":
```

```
    group = input(
        "Group: "
    )
```

```
    result = filter_by_group(
        students,
        group,
    )
```

```
    print_students(result)
```

```
elif command == "4":
```



```
best = find_best_student(  
    students,  
)  
  
if best:  
    print(  
        "Best:",  
        best.full_name,  
        best.average_grade,  
    )  
  
elif command == "5":  
    print("Goodbye.")  
    break  
  
else:  
    print(  
        "Unknown command."  
    )
```

25. Порядок виконання лабораторної роботи

Встановити Python.

Створити каталог проєкту.

Створити .venv.

Активувати virtual environment.

Ініціалізувати Git repository.

Створити .gitignore.

Створити src-layout.

Створити package.

Створити models.py.

Створити services.py.

Створити main.py.

Реалізувати свій індивідуальний варіант.

Додати type hints.

Створити pyproject.toml.

Встановити package у editable mode.

Перевірити запуск.

Створити README.md.

Зробити не менше трьох Git commits.

Перевірити git status.

Сформувати звіт.

26. Контрольні запитання

Що таке Python-проект?

Чим проект відрізняється від одного Python script?

Що таке Python module?

Що таке Python package?

Для чого потрібний __init__.py?

Що таке src-layout?

Які переваги має src-layout?

Для чого використовується virtual environment?

Як створити .venv?

Як активувати virtual environment?

Для чого використовується pip?

Чому рекомендується python -m pip?

Для чого потрібний pyproject.toml?

Що містить секція [project]?

Що таке editable installation?

Для чого використовується if __name__ == "__main__"?

Що таке Git?

Що таке repository?

Що таке commit?

Для чого використовується git status?

Для чого git add?

Для чого git commit?

Що показує git log?

Для чого потрібен .gitignore?

Чому .venv не потрібно комітити?

Для чого потрібний README?

Що таке type hints?

Для чого програму розділяють на modules?

Що таке business logic?

Чому потрібно відокремлювати application entry point від business logic?

27. Вимоги до звіту

Звіт повинен містити:

титульну сторінку;

номер і тему лабораторної роботи;
мету;
номер індивідуального варіанта;
постановку задачі;
структуру проєкту;
опис створених modules;
текст основних фрагментів коду;
пояснення pyproject.toml;
вміст .gitignore;
фрагмент README;
screenshots запуску application;
результат git status;
результат git log --oneline;
відповіді на контрольні запитання;
висновок.

28. Критерії оцінювання

Максимальна оцінка — 4 балів.

Компонент | Бали

Правильна структура src-layout | 0.4

Virtual environment та pyproject.toml | 0.4

Коректна реалізація індивідуального завдання | 1.2

Модульність та якість Python-коду | 0.4

Type hints та стиль коду | 0.4

Git та логічна історія commits | 0.4

README і документація | 0.4

29. Висновок

У лабораторній роботі студент переходить від створення окремих Python scripts до організації повноцінного програмного проєкту. Використання src-layout, modular architecture, virtual environment, pyproject.toml, Git та README формує фундамент для подальших лабораторних робіт.

Проєкт, створений у лабораторній роботі №1, доцільно використовувати як основу наступних робіт курсу. У наступних лабораторних роботах до нього можуть поступово додаватися складні структури даних, ООП, type checking, tests, persistence, API, asynchronous operations, profiling, Docker та CI/CD.